

# Semi-Automated, Open-Sourced Utility Network Reconstruction using LiDAR and OpenStreetMaps

1<sup>st</sup> Alyssa Acquaviva

*National Laboratory of the Rockies*  
Fairbanks, AK

2<sup>nd</sup> Al-Idrisi Haddad

*National Laboratory of the Rockies*  
Fairbanks, AK

3<sup>rd</sup> Owen Tanuwidjaja

*National Laboratory of the Rockies*  
Golden, CO

4<sup>th</sup> Deepthi Vaidhynathan

*National Laboratory of the Rockies*  
Golden, CO

5<sup>th</sup> Kumaraguru Prabakar

*National Laboratory of the Rockies*  
Golden, CO

6<sup>th</sup> David Martinez Biro

*National Laboratory of the Rockies*  
Fairbanks, AK

**Abstract**—Open-access LiDAR offers a promising resource for mapping distribution-level electrical infrastructure, particularly in remote regions where traditional surveying is costly and utilities cover vast service territories. However, publicly available point-cloud datasets often contain noise, incomplete returns, and heterogeneous sampling densities, making reconstruction of poles and network connectivity a challenging remote-sensing task. This work presents a semi-automated method that integrates NOAA’s open-access LiDAR with OpenStreetMap (OSM) power-line annotations to identify pole locations, infer conductor height, and estimate network connections without requiring proprietary data or on-site surveys. Unguided Random Sample Consensus (RANSAC) and clustering techniques are applied to pole-centered point-cloud subsets to extract conductor information. In a 1 km<sup>2</sup>, singular LiDAR tile test site in southeast Alaska, the model achieved 70% accuracy in pole connection mapping compared to OSM. An unguided clustering test further showed that conductor counts were predicted within one conductor for 13 of 15 poles. By harmonizing heterogeneous open-access datasets, this pipeline demonstrates a replicable framework for cross-domain data integration, transforming raw geospatial returns into interoperable network models reusable across engineering disciplines. **Index Terms**—LiDAR, network modeling, data reuse

roofs and simple polygons to train convoluted neural network models, though authors note the limitations of expensive LiDAR surveys. [2]. RANSAC-based clustering methods with catenary line fitting have shown promise for differentiating conductors [3], though reconstruction quality is not yet sufficient for fully automated workflows [4]. This work addresses these gaps using two publicly available datasets: NOAA’s Digital Coastal LiDAR [5] and OpenStreetMap power-line geometries [6]. With over 33 trillion LiDAR points concentrated in coastal areas where more than half of the U.S. population lives, and OSM expanding power network coverage, these sources offer broad applicability [7]; [8]. The fusion of these sources demonstrates a framework for data integration in which freely available datasets from distinct disciplines are transformed and linked into interoperable network models, contributing a replicable model for geospatial data reuse. This work proposes two primary contributions: (1) a reproducible, open-source pipeline integrating heterogeneous datasets (airborne LiDAR and crowdsourced OSM geometries) into network models suitable for power-flow modeling; and (w) a comparative evaluation of DBSCAN+PCA and sequential RANSAC for conductor extraction from sparse airborne point clouds.

## I. INTRODUCTION

Utility service territories in extreme environments highlight a compelling use case for airborne LiDAR datasets, but also expose the limitations of open-access point clouds which are often noisy, sparse, and heterogeneous in sampling density, making reconstruction a challenging remote-sensing task. Many remote communities and utilities lack high fidelity data to enable informed decision-making for electrification and local generation, making power flow studies highly valuable, but costly and resource-intensive. Accurately reconstructing conductor configuration, spacing, and height enables impedance estimation and power-system modeling.

LiDAR point-cloud data has enabled 3D reconstruction of electrical infrastructure over large areas [1], and prior work has augmented OpenStreetMap with LiDAR for urban mapping of

## II. METHODOLOGY

### A. Experimental Approach and Data Sourcing

The objective of this work is to automatically extract utility pole attributes in an Area of Interest (AOI) located in a community in southeast Alaska, including pole heights, conductor count, and pole connections, from LiDAR and OpenStreetMap (OSM) data. The extracted information is intended for direct use in future power flow simulations. All scripts were implemented in Python 3.11 using open-sourced libraries.

The power system infrastructure at the site is partially available on OpenStreetMap, including poles and powerlines. NOAA’s 2018 Coastal LiDAR dataset provides high-resolution point cloud data that enables validation and enhancement of this open-source geospatial information. The State of Alaska Division of Geological and Geophysical Surveys (DGGS)

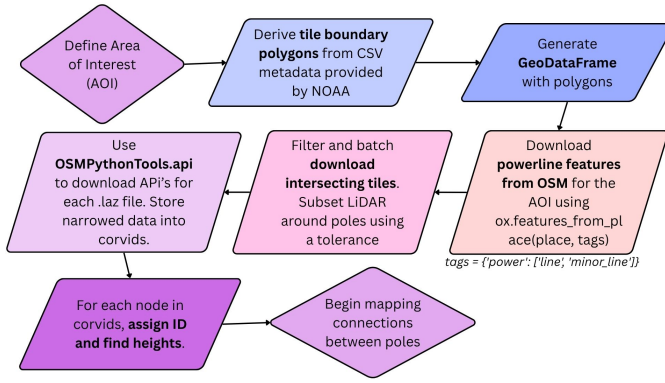


Fig. 1. Overview of data preprocessing workflow. Power line geometries and pole node coordinates are retrieved from OpenStreetMap.

produced the data in April 2018. NOAA Office of Coastal Management downloaded this dataset, released with an open end-user license and hosted on their website. For the selected area, the LiDAR dataset was in an orthometric vertical datum North American Vertical Datum of 1988 (NAV88) using GEOID12B.

1) *Geospatial Data Processing*: To isolate only the LiDAR data tiles relevant to the target study area, a spatial filtering routine was implemented as demonstrated in figure 1, using CSV metadata from NOAA. First, pole locations were obtained from OpenStreetMap (OSM) using both the `osmnx` and `OSMPythonTools` Python libraries. All features tagged with `power = line` or `power = minorline` were retrieved, and their associated node coordinates, representing individual utility poles, were extracted. `row_to_polygon` and `GeoDataFrame(raw range)` convert tile metadata into polygons, which are intersected with a buffered convex hull around pole coordinates. Poles were located using their geographic coordinates from OSM, and only the 26 .laz files overlapping those pole locations were selected for automated download using the Python requests and pathlib libraries. Each tile was then spatially subset using a custom `constrain()` function that isolates a local window around each pole coordinate, reducing computational load.

### B. Extract Pole-Centered Point Cloud Features

Each relevant .laz file was read using the `laspy` library in Python and converted from its stored integer-based coordinate system into metric units using the file's internal scaling factors. Power infrastructure data were then retrieved from OpenStreetMap via the `OSMPythonTools` API, which provides both the linear geometries of powerlines and their constituent node coordinates. Each node represents a utility pole and includes a unique OSM identifier. These pole coordinates and node IDs were extracted and stored in a list, forming the reference dataset used to spatially anchor each pole's local LiDAR subset.

For each pole coordinate, two local point cloud subsets were extracted (1) A broad region ( $\text{radius} = \text{tol}$ ) to capture the full pole and attached conductors,  $\text{tol} = 0.00003 - 0.000008^\circ$ ,

and (2) A narrower region ( $\text{radius} = \text{tol}/3$ ) to estimate the local ground elevation using only points classified as ground returns. Pole height was computed as the difference between the 98th percentile of the full subset (the top of the pole) and the 5th percentile of the ground subset. These percentile was chosen instead of the absolutes to mitigate outliers such as misclassified vegetation or aerial noise. For each valid pole, spatial attributes including estimated ground elevation, top elevation, geographic coordinates, and associated OSM node ID, were recorded.

### C. Derive Pole Network Connections

Once pole features were extracted, potential connections between poles were inferred using both OpenStreetMap data and geometric proximity analysis. OSM connection data serves as a reference for validation; in regions lacking OSM coverage, this algorithm would operate without ground truth, which is its primary intended use case. Inferred connections are computed via Haversine distance and bearing-angle filtering. For the area of interest, a maximum distance between poles of 120m was used in order to filter out extraneous connections while capturing the longest evident runs. Then, bearing angles between coordinates were calculated. All valid connections were then filtered by bearing angle. A minimum angle to be considered a valid connection was  $55^\circ$ .

### D. Clustering Point Cloud Data

As a proof of concept, a simple algorithm was used to cluster LiDAR point cloud data into clusters that resembled conductors in order to automate the designation of pole attributes in a region of interest. To generate an estimate of the number and height of conductors per pole, all point cloud data per .laz was visualized in Open3D and a clustering algorithm was employed to organize points of similar height and co-linearity. To isolate conductors, points within the upper 6–7 m of each pole were analyzed after removing the pole core ( $\geq 5\text{--}10$  cm radius). Two clustering methods were evaluated. The DBSCAN + PCA method groups points by height and planimetric distance, with adaptive parameters estimated from nearest-neighbor spacing and eigenvalue ratios derived by PCA. The RANSAC method replaces spatial clustering with iterative line fitting in the XY-plane, extracting conductor segments one at a time while removing inliers after each iteration. Lines are accepted if their span exceeds the minimum line length and their residual fit quality exceeds the allowable confidence. Each candidate conductor is assigned a confidence score based on point density and linearity. This process yields both a count of visible conductors and their mean heights above ground for comparison across clustering techniques.

## III. PROOF OF CONCEPT

The geospatial preprocessing, pole feature extraction, and conductor clustering routines were applied on an available LiDAR dataset for an AOI located in south-east Alaska. Considering the quantity of data, this proof of concept will focus on validating one .laz file capturing approximately 1 square

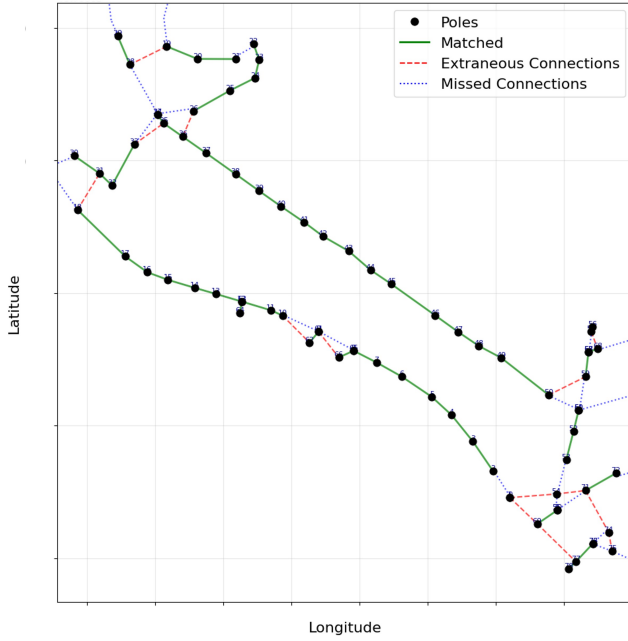


Fig. 2. The known OpenStreetMap connections in the area of 1 .laz file is overlaid with predicted connections found from Haversine + Angle based filtering. Short, angular connections tend to be misrepresented over connections of larger distances.

kilometer. Using the mapping routine, 79 OSM-referenced poles were processed and 75 valid electrical pole connections were found that met the filtering criteria. Figure 2 show how one .laz file performs against known OSM connections. The algorithm correctly predicts about 70% of the connections. Accuracy here is defined as the number of inferred connections present in OSM divided by the total number of inferred connections. It very accurately predicts the number of connections, but over small distances, the algorithm tends to inaccurately predict which poles are connected, resulting in mismatched connections.

Two clustering methods are compared identify feasibility of conductor identification: DBSCAN + PCA and RANSAC. Both the DBSCAN and RANSAC algorithms sequentially search each pole's point cloud for clusters, meaning that parameters like minimum line length and maximum perpendicular deviation are first set to conservative values to eliminate noise, better capturing lines. Then, if insufficient conductors are captured with these parameters, they are relaxed and the loop is run again. The parameters for both methods are listed below in table I. In general, the RANSAC method was more robust, allowing parameters to be stricter to better filter out noise. In contrast, the DBSCAN struggled to capture points that clearly belonged to a conductor, therefore parameters needed to be wider in order to capture these points, resulting in missed or extra conductors.

Algorithms like RANSAC are capable of fitting lines automatically and sequential fitting helps find multiple lines even if they overlap in space. DBSCAN groups points based

| PARAMETER              | DBSCAN + PCA                          | RANSAC             |
|------------------------|---------------------------------------|--------------------|
| $tol_{deg}$            | $6 \times 10^{-5} - 8 \times 10^{-5}$ | $6 \times 10^{-5}$ |
| Min Samples Z          | 2                                     | 3                  |
| Min Samples XY         | 2                                     | NA                 |
| Min Points per cluster | 2                                     | NA                 |
| Confidence             | 0.50                                  | 0.65               |

TABLE I

KEY PARAMETERS TO FIT CLUSTERS BY METHOD FOR A PROOF OF CONCEPT. SPATIAL TOLERANCE IS IN THE EPSG:4326 DEGREES. AT THE STUDY LATITUDE,  $1 \times 10^{-5} = 1.1m$ .

| POLE ID      | GOOGLE COUNT | RANSAC COUNT | DBSCAN COUNT |
|--------------|--------------|--------------|--------------|
| 6            | 4            | $4^{B,M}$    | $3^{B,M}$    |
| 7            | 5            | 5            | $3^M$        |
| 8            | 5            | $6^V$        | $4^{V,M}$    |
| 9            | 4            | 4            | $1^M$        |
| 10           | 5            | 5            | $4^M$        |
| 11           | 4            | 4            | $2^M$        |
| 12           | 5            | 5            | $7^B$        |
| 13           | 5            | $6^B$        | $4^M$        |
| 14           | 5            | $6^B$        | 5            |
| 15           | 5            | $6^V$        | $5^{M,V}$    |
| 4b           | 3            | 3            | $6^B$        |
| 5b           | 4            | 4            | $5^B$        |
| 6b           | 4            | 4            | $6^{B,V}$    |
| 8b           | 4            | $6^{V,X}$    | $6^V$        |
| 13b          | 1            | 1            | $6^V$        |
| Avg per pole | 4.47         | 4.6          | 4.47         |

TABLE II

FOR 15 POLES, CONDUCTOR COUNTS ARE COMPARED BY METHOD TO THE NUMBER OF POLES AS COUNTED ON GOOGLE MAPS. POLES TAKEN FROM 2 SEPARATE .LAZ FILES. ERRORS: B = CONDUCTOR BROKEN INTO SEGMENTS, X = INFRASTRUCTURE INADVERTENTLY CLUSTERED, V = VEGETATION INADVERTENTLY CLUSTERED, M = MISSING CONDUCTOR

on density and proximity, making it an appropriate tool to cluster point cloud data by height. DBSCAN, RANSAC, and PCA in different combinations were used, primarily using the `Scikit-learn` library. Table II shows the estimated number of conductors for 10 poles by method compared to the real pole, found on Google Maps.

DBSCAN+PCA struggled with sparse, unevenly sampled point clouds: wide XY spacing merged parallel conductors while tight spacing fragmented single conductors, and PCA failed to split closely-spaced parallel lines at low point densities. The RANSAC method is much more accurate, but still clusters vegetation and breaks some conductors into segments. From this snapshot, the RANSAC algorithm appears to slightly overestimate conductor count, but is within one conductor of the true count most of the time. Exact matches occurred in 9 out of the 15 poles analyzed, and an additional 4 poles were within 1 conductor. Additionally, this accuracy was improved by adjusting the tolerance between .laz files manually, reducing impacts from vegetation in certain areas of town. Figure 3 shows the outcome of the RANSAC algorithm for one pole (pole 10 in table II) with the parameters. Each color represents one conductor. Lengthening the minimum length requirement per line could help reduce broken conductors, but may inadvertently skip conductors with few points, even with

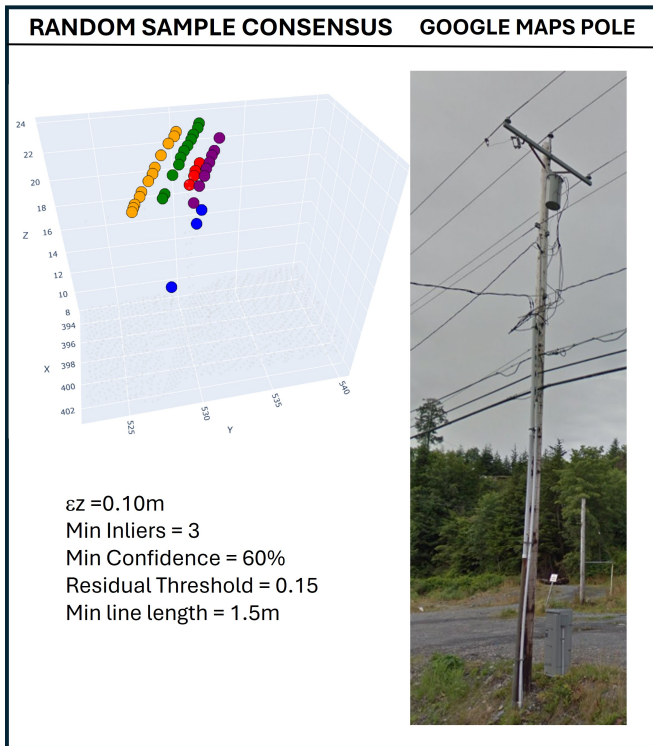


Fig. 3. Pole 10, clustered using the RANSAC algorithm, compared to its Google Maps counterpart. The RANSAC algorithm successfully clusters 5 lines to match the image, ignoring lighting and communication cables.

a sequential approach. The RANSAC routine could be further improved by cross-referencing a pole's conductors with that of the connecting poles.

The primary output of this routine is a JSON file containing the pole height, number of conductors, and the conductors' heights for all poles in the downloaded .laz files, formatted for direct ingestion into power flow modeling software via conversion into an IEEE common format file. The latitude and longitude of the proof-of-concept study area are withheld to respect community sensitivity. All methodological steps, algorithms, and parameter settings are provided in full, enabling replication of the workflow with any suitable LiDAR and OpenStreetMap dataset.

#### IV. CONCLUSIONS

This work presents a reproducible, open-source workflow for LiDAR-based electrical infrastructure assessment. For any location with sufficient NOAA LiDAR coverage, the method automatically identifies and downloads relevant .laz tiles, detects pole structures, estimates heights, infers network connections, and estimates conductor counts, providing sufficient geometric data for use in power flow modeling. Employing this approach, grid modelers can reduce manual effort compared to site visits or GIS-coded image interpretation. Beyond network mapping, the method has broader applications: year-on-year height comparisons could detect infrastructure shifts from permafrost thaw, or vegetation creep and conductor geometry outputs support impedance estimation for evaluating the

impact of new community-scale generation or large industrial loads on power quality and resilience.

Primary limitations include dependence on OSM pole coordinates as anchor points, meaning the pipeline performs attribute extraction at known locations rather than blind detection, and sensitivity to vegetation, particularly in the forested terrain common to the remote communities this work targets. Hard coded values such as the 120m distance cap and 55° bearing filter were empirically set for this site; while RANSAC's iterative inlier fitting reduces sensitivity to the line-level parameters, the network-level filters would require retuning for sites with different pole spacing or street geometry. Future directions include catenary sag modeling, multi-scale processing, and integration with power-flow modeling software. All scripts were implemented in Python 3.11 using open-source libraries (laspy, osmnx, GeoPandas, Shapely, Open3D, Scikit-learn, and requests). A reproducible Jupyter Notebook is available upon request.

#### ACKNOWLEDGMENT

The research was performed using computational resources sponsored by the Department of Energy's Office of Critical Minerals and Energy Innovation and NLR. This work was authored by NLR for the U.S. Department of Energy (DOE), operated under Contract No. DE-AC36-08GO28308. This work was supported by the Laboratory Directed Research and Development (LDRD) Program at NLR. The views expressed in the article do not necessarily represent the views of the DOE or the U.S. Government. The U.S. Government retains and the publisher, by accepting the article for publication, acknowledges that the U.S. Government retains a nonexclusive, paid-up, irrevocable, worldwide license to publish or reproduce the published form of this work, or allow others to do so, for U.S. Government purposes.

#### REFERENCES

- [1] R. McLaughlin, "Extracting transmission lines from airborne lidar data," *IEEE Geoscience and Remote Sensing Letters*, vol. 3, no. 2, pp. 222–226, 2006.
- [2] M. Barranquero, A. Olmedo, J. Gómez, A. Tayebi, C. J. Hellín, and F. Saez de Adana, "Automatic 3d building reconstruction from openstreetmap and lidar using convolutional neural networks," *Sensors*, vol. 23, no. 5, 2023. [Online]. Available: <https://www.mdpi.com/1424-8220/23/5/2444>
- [3] N. Munir, M. Awrangjeb, and B. Stantic, "3d reconstruction of bundle sub-conductors using lidar data from forest terrains," in *IGARSS 2022 - 2022 IEEE International Geoscience and Remote Sensing Symposium*, 2022, pp. 7230–7233.
- [4] "Shape-based recognition of 3D point clouds in urban environments | IEEE Conference Publication | IEEE Xplore." [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/5459471>
- [5] NOAA, "Coastal Topographic Lidar." [Online]. Available: <https://coast.noaa.gov/digitalcoast/data/coastallidar.html>
- [6] O. Contributors, "Open Street Map." [Online]. Available: <https://www.openstreetmap.org/copyright>
- [7] G. Boeing, "Modeling and analyzing urban networks and amenities with osmnx," *Geographical Analysis*, vol. n/a, no. n/a, 2025. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1111/gean.70009>
- [8] T. D.-W. Sen. Baldwin, "Text - S.1069 - 116th Congress (2019-2020): Digital Coast Act," Dec. 2020, archive Location: 2019-04-09. [Online]. Available: <https://www.congress.gov/bill/116th-congress/senate-bill/1069/text>